项目背景

一、宏观趋势：旅游消费井喷，政策力推 AI 导览

我国旅游业正处于强劲复苏与高速增长期。据文化和旅游部数据，2025 年国内居民出游 65.22 亿人次，同比增长 16.2%，出游总花费 6.30 万亿元；较 2024 年的 56.15 亿人次（同比 +14.8%）再创历史新高。游客规模逐年攀升，景区接待压力与服务需求同步激增。

与此同时，国家政策明确指向智慧化。2024 年 5 月，文化和旅游部等五部门联合印发《智慧旅游创新发展行动计划》，明确鼓励支持旅游景区探索"人工智能导游"等智慧旅游创新服务；同年全国智慧旅游解决方案遴选将"智慧导览、智慧讲解"列为重点方向。我国智慧旅游经济规模已从 2019 年的 43.29 亿元增至 2023 年的 103.82 亿元，五年增长 1.4 倍。

一句话：游客越来越多、要求越来越高，而传统人工导览服务的供给与质量却难以匹配——这一供需矛盾，正是国家以政策力推 AI 导游的根本原因，也是本项目的立项起点。

二、传统景区导览服务的核心痛点

在旅游人次年年攀升的趋势下，传统景区导览服务面临以下痛点（按"讲解供给侧—游客侧—管理侧"三类主体归纳）：

讲解供给侧

痛点 1·真人讲解资源不足。 全国约 66 万持证导游，但实际与旅行社签订劳动合同从业者仅约 16.5%；旺季与热门景区导游供不应求，存在明显的局部性缺口。游客"一进景区找不到讲解、跟不上团"成为普遍现象，难以满足日益增长的高品质讲解需求。

痛点 2·讲解内容同质、缺乏个性化。 真人讲解多为"一套词讲所有人"，无法依据每位游客的同行关系（亲子/老人/情侣）、兴趣偏好与停留节奏提供差异化讲解，个性化严重不足。

痛点 8·多语种与深度讲解能力稀缺。 入境游持续回暖，但外语导游仅占持证导游的约 8.4%，小语种导游长期"无人可用"；同时能提供深度文化讲解的专家型导游紧缺。国际游客与深度文化体验需求难以被满足。

游客侧

痛点 4·缺少全局人流感知与智能游览规划。 游客无法感知景区整体客流分布，常出现"某景点人挤人、另一些景点却人少"的失衡，既影响体验又埋下安全隐患。游客亟需基于实时客流的智能游览顺序规划与错峰建议。

痛点 5·游览过程中的情绪起伏无人关照。 排队、迷路、天气不适等情形下，游客难免产生焦躁、失落等负面情绪。此时一句及时的安抚与关怀，能提供可观的情绪价值，而传统导览完全缺位。

痛点 6·到达景点后缺少即时、按需的针对性讲解。 游客抵达某一景点时，往往需要"专人即时讲解"，且应区分游客身份（如带孩子需讲故事、老人需放慢节奏）作个性化输出——这是真人导游难以规模化覆盖的。

管理侧

痛点 3·游客反馈难采集、服务质量难量化。 景区管理方缺乏方便、直接的手段及时了解游客真实感受，难以量化评估导览服务质量，导致无法形成"反馈—改进"闭环，后续游客体验难以持续优化。

痛点 7·突发情况难以及时感知与上报。 景区人流密集，拥挤、求助、投诉等突发情况不可避免，传统模式下缺乏从游客端到管理端的实时反馈通道，管理方难以第一时间发现并处置。

三、本项目的回应

针对上述痛点，本项目构建景区导览服务 AI 数字人系统，以"可 7×24 服务、零边际成本、可个性化、可量化"的 AI 导游，系统性地回应：

- 对痛点 1/2/6/8——以 RAG 智能问答 + 多同行人格 + 多语言，提供无限量、个性化、可按身份差异化、支持多语种的专业讲解；
- 对痛点 4/5——以 实时人流感知分流 + 时间天气感知 + 情绪安抚意图，主动给出错峰路线与情绪关怀；
- 对痛点 3/7——以 会话情感分析 + 数据大屏 + 游客感受度报告 + 负面情绪/突发预警，为管理方建立"实时感知—量化评估—闭环优化"的能力。

更关键的是，系统采用插拔式知识库架构——导入任意景区知识库即可复用全部能力，不绑定单一景区，具备规模化推广价值。

数据来源

- [文化和旅游部：2025 年国内居民出游 65.22 亿人次（新华网）](https://www.news.cn/20260126/8dda85c98d8843e69b256cea591a7d8b/c.html)
- [2024 年度国内旅游数据情况（文化和旅游部）](https://zwgk.mct.gov.cn/zfxxgkml/tjxx/202501/t20250121_958012.html)
- [《智慧旅游创新发展行动计划》（中国政府网）](https://www.gov.cn/zhengce/zhengceku/202405/content_6950881.htm)
- [2024 年全国智慧旅游解决方案推荐遴选通知（文化和旅游部）](https://zwgk.mct.gov.cn/zfxxgkml/zykf/202410/t20241009_955624.html)
- [导游证报考激增，存在局部性缺口（新京报）](https://www.bjnews.com.cn/detail/1733916660129950.html)
- [月入两万不及以前一半，小语种导游仍然"无人可用"（大众日报）](https://m.dzplus.dzng.com/share/general/0/NEWS1128806CCXHYWVXDLEVX)
- [景区敞开大门迎客，旅游体验如何保障（新华网）](http://www.news.cn/politics/20240625/6a9f5cf8fd524468a5548068e40f188a/c.html)

图 2-1　系统用例图

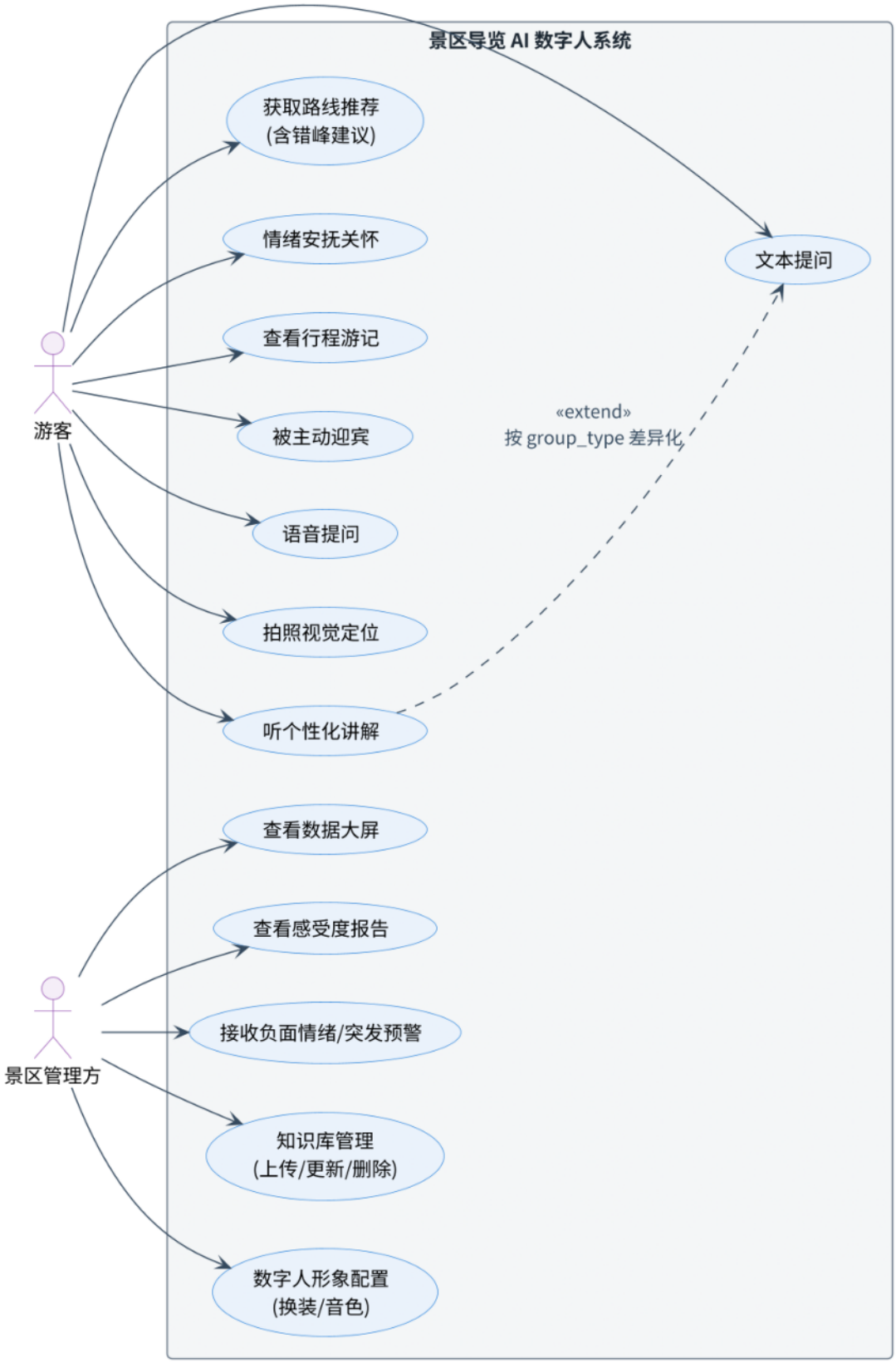


图 2-1 系统用例图

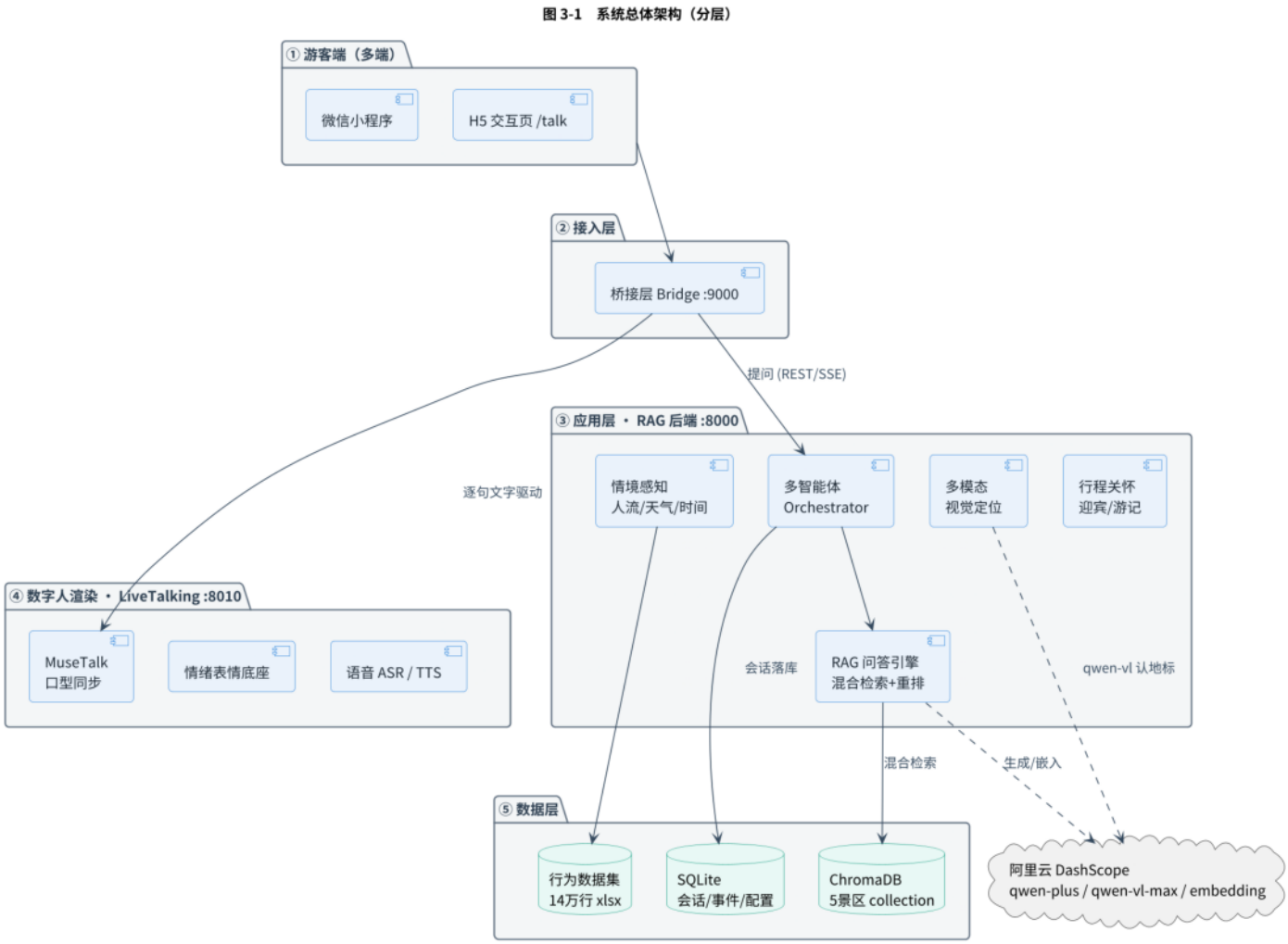

图 3-1 系统总体架构（分层）
① 游客端（多端）
微信小程序
H5 交互页 /talk
② 接入层
桥接层 Bridge :9000
提问 (REST/SSE)
③ 应用层 · RAG 后端 :8000
情境感知
人流/天气/时间
多智能体
Orchestrator
多模态
视觉定位
行程关怀
迎宾/游记
逐句文字驱动
④ 数字人渲染 · LiveTalking :8010
MuseTalk
口型同步
情绪表情底座
语音 ASR / TTS
会话落库
RAG 问答引擎
混合检索+重排
qwen-vl 认地标
混合检索
生成/嵌入
⑤ 数据层
行为数据集
14万行 xlsx
SQLite
会话/事件/配置
ChromaDB
5景区 collection
阿里云 DashScope
qwen-plus / qwen-vl-max / embedding

图 3-1 系统总体架构

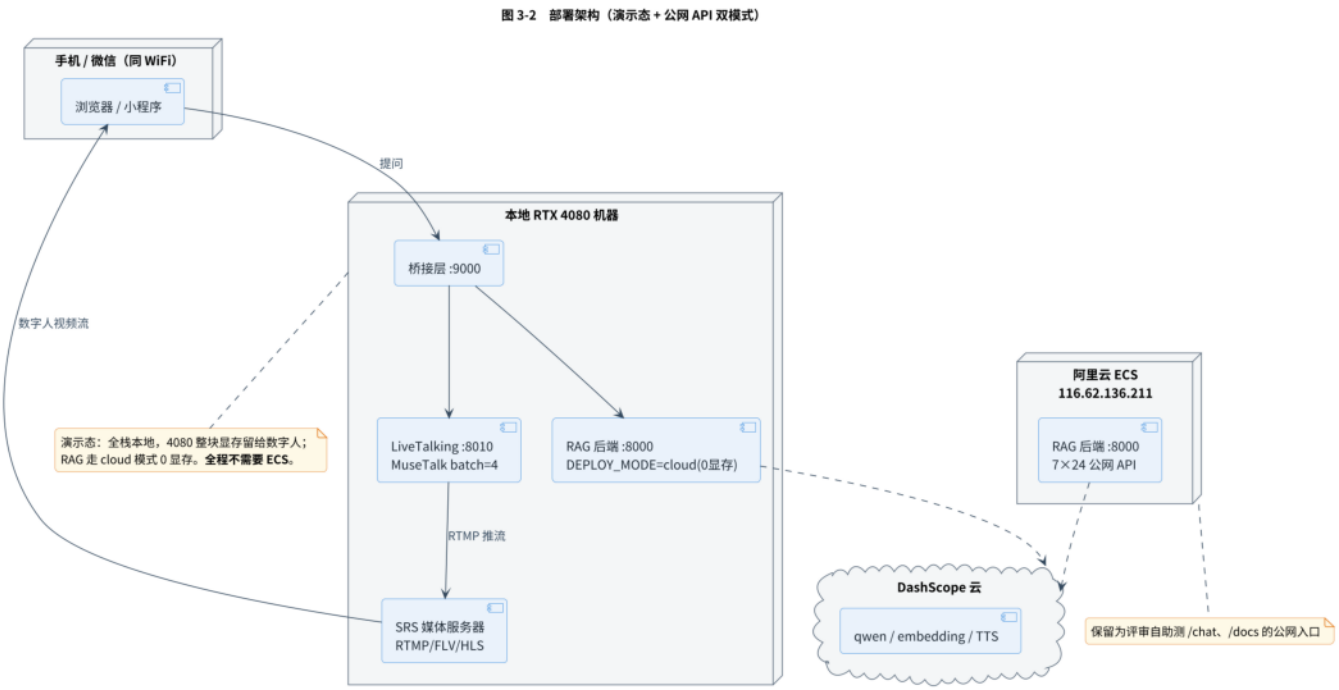

图 3-2 部署架构（演示态 + 公网 API 双模式）
手机 / 微信（同 WiFi）
浏览器 / 小程序
提问
本地 RTX 4080 机器
桥接层 :9000
数字人视频流
演示态：全栈本地，4080 整块显存留给数字人；
RAG 走 cloud 模式 0 显存。全程不需要 ECS。
LiveTalking :8010
MuseTalk batch=4
RAG 后端 :8000
DEPLOY_MODE=cloud(0显存)
RTMP 推流
SRS 媒体服务器
RTMP/FLV/HLS
阿里云 ECS
116.62.136.211
RAG 后端 :8000
7×24 公网 API
DashScope 云
qwen / embedding / TTS
保留为评审自助测 /chat、/docs 的公网入口

图 3-2 部署架构

第 X 章　算法设计：RAG 智能问答引擎

本章是全文的技术核心。景区导览的"智能问答"能力，本质是一个检索增强生成（Retrieval-Augmented Generation, RAG）系统：让大模型在回答前先从景区知识库中检索出相关事实，"开卷作答"而非"凭记忆瞎猜"。本章按"总体框架 → 基座选型 → 逐层原理 → 工程优化 → 质量评测"的顺序展开，论证我们如何把准确率做到 97.1%、把首句延迟压到 1.8s，并保证这套能力对任意景区知识库通用。

X.1　为什么是 RAG，而不是直接问大模型

大模型（如通义千问）虽博闻强记，但直接用于景区问答有三个致命缺陷：

- 痛点一·知识盲区：大模型不可能知道"五明桥的尺寸""佛足坛的规模"这类冷门、细粒度的景区事实——这些不在它的预训练语料里。
- 痛点二·幻觉：被问到不确定的信息时，大模型倾向编造一个看似合理的答案（如把灵山大佛说成"约 101 米"），对导游场景是致命的。
- 痛点三·无法更新：景区信息会变（票价、开放时间），重新训练大模型成本极高且不现实。

RAG 用一个优雅的思路同时解决三者：把景区知识存进可检索的外部知识库，回答前先检索、再把检索结果作为"参考资料"喂给大模型。知识盲区由检索补全，幻觉由"只许照资料回答"约束，更新只需替换知识库文档。这也正是赛题"插拔式知识库"要求的技术落点——换景区 = 换知识库，算法链路零改动。

本章用一组消融实验直接量化了 RAG 的价值（X.8.1）：关闭检索后，同一系统的准确率从 97.1% 暴跌至 41.0%。这 56 个百分点，就是 RAG 的贡献。

X.2　总体框架

系统的问答全链路如下（图 X-1）：

```
游客提问 query
   │
   ├──[0] 多智能体路由 Orchestrator ── 判定意图(qa/导航/推荐/安抚/闲聊)，决定处理策略
   │
   ├──[1] 答案缓存 LRU ── 命中热门问题直接返回(<5ms)，跳过后续全部计算
   │
   ├──[2] Query Rewrite ── 含指代词("它多高")时改写为检索友好形式("灵山大佛高度")
   │
   ├──[3] 混合检索 Hybrid Retrieval
   │       ├── 稠密检索：BGE-M3 把 query 编码为 1024 维向量，在 ChromaDB 按余弦相似度取 Top-10
   │       ├── 稀疏检索：BM25 按关键词精确匹配取 Top-10
   │       └── 融合去重 → 候选集(约 14 条)
   │
   ├──[4] 重排序 Rerank ── BGE-Reranker-v2-m3 对候选逐条精算相关度，取 Top-4
   │
   ├──[5] Prompt 组装 ── 知识库片段 + 导游人格 + 意图指令 + 情境上下文(客流/天气)
   │
   ├──[6] LLM 生成 ── qwen-plus 流式生成，逐句输出
   │
   └──[7] 情绪解析 + 落库 ── 抽取 [情绪:X] 驱动数字人表情，对话存入 SQLite
```

设计理念一句话：以"混合检索召回 + 交叉编码精排"保证找得准，以"Prompt 强约束 + 召回兜底"保证不乱说，以"LRU 缓存 + 并行检索 + 逐句流式"保证响应快。

X.3　基座选型：每个组件都"为何适合本任务"

算法系统的可信度，首先取决于基座选型是否经得起推敲。我们的每个组件都遵循"是什么 → 为何适合本任务 → 数据/事实佐证"的论证范式，而非简单调包。

X.3.1　向量嵌入模型：BAAI/BGE-M3

| 候选 | 取舍 |

|---|---|
OpenAI text-embedding	需海外 API、有费用、数据出境合规风险
text2vec / m3e	中文尚可，但多语种与长文本偏弱
BGE-M3（选定）	智源开源，中英双语检索质量领先、支持 8192 长上下文、可本地部署零 API 费用；1024 维稠密向量

为何适合本任务：景区知识含大量中文专名与少量外文（面向国际游客），BGE-M3 的中英双语能力直接命中；本地运行（RTX 4080）则规避了 API 费用与合规问题。部署到无 GPU 的云服务器时，无缝切换为效果相近的 DashScope text-embedding-v3（deploy_mode 一键切换）。

X.3.2　重排序模型：BAAI/BGE-Reranker-v2-m3

这是把准确率从"能用"推到"90%+"的关键一环，原理见 X.5。选型理由：与 BGE-M3 同源、中英双语、交叉编码精度高、可本地 fp16 运行（半精度，省显存提速）。

X.3.3　生成大模型：qwen-plus（事实问答）+ qwen-turbo（生成性意图）

这是一处经过实测权衡的工程决策，而非随手选型：

模型	生成速度	用途	理由
qwen-max	~20 字/s	（备选）	质量最高但慢，流式首句 ~3s
qwen-plus（选定·qa）	~48 字/s	事实问答	速度约 2 倍于 max，流式首句 ~1.2s，是压住语音 <5s 的关键；事实已由 RAG 检索结果约束，质量足够
qwen-turbo（选定·生成性）	最快	路线/推荐/安抚/闲聊	这些意图不在准确率测试集范畴，更看重生成流畅与速度

核心洞察：在 RAG 架构下，事实正确性由检索保证，大模型只负责组织语言。因此可以放心地为生成模型选择"更快而非最强"的档位——这是 RAG 解耦"知识"与"表达"后释放出的优化空间。

X.4　逐层原理（上）：知识库构建

RAG 的质量从知识"入库"那一刻就开始决定。

X.4.1　文档解析与"双策略"分块

知识库文档有两种形态，我们针对性地采用不同分块策略（backend/rag/loader.py）：

- 叙述性文档（《游览指南》）：按段落聚合，相邻 chunk 保留 128 字重叠（chunk 大小 512），避免把一句话从中切断导致语义断裂。
- 结构化表格（《景点结构化数据集》）：每一行（一个景点）独立成块，且把单元格拼成"字段名: 值"格式（如"建筑参数: 长 39.8m，高 7m"）。

踩坑与解法：早期把表格按普通文本流式分块，导致"灵山大佛的高度"可能与"梵宫的面积"被切进同一 chunk，检索时张冠李戴。改为"逐行成块 + 字段名保留"后，每个景点的事实自成一个语义完整、互不污染的检索单元——这是结构化景区数据准确率高的隐性功臣。

X.4.2　向量化与命名空间隔离

每个 chunk 经 BGE-M3 编码为 1024 维归一化稠密向量，存入 ChromaDB。关键设计是多 collection 命名空间隔离：每个景区独占一个 collection（scenic_lingshan、scenic_xihu……），检索时只在目标景区的命名空间内进行。这从数据层面实现了"插拔式知识库"——新增景区 = 新建一个 collection，互不干扰，也杜绝了 X.8 陷阱测试中"跨景区串问"的污染。

X.5　逐层原理（下）：检索-重排-生成

X.5.1　混合检索：稠密召回语义，稀疏兜底专名

单一检索方式都有盲区，我们双路并行（backend/rag/retriever.py）：

- 稠密检索（语义）：query 经 BGE-M3 编码，在 ChromaDB 中以余弦相似度召回语义最近的 Top-10。它能理解"大佛多高"≈"佛像高度"这类同义改写，但对生僻专名不敏感。
- 稀疏检索（关键词）：BM25 算法按词频精确匹配召回 Top-10。它对"九龙灌浴""鲫鱼背"这类专有名词精准命中——而这恰是稠密向量的弱项。

工程细节：中文 BM25 我们采用字符级切分（而非依赖分词器），既规避了分词错误，又对未登录的景区专名天然鲁棒。

两路结果去重融合为约 14 条候选。"语义 + 关键词"互补，是召回率（不漏）的保障。

X.5.2　重排序：从"召回"到"精排"的关键一跃

混合检索保证了正确答案在候选集里（高召回），但候选集里也混着噪声，且顺序不一定理想。重排序解决"把最相关的排到最前"（高精度）。

这里用到交叉编码器（Cross-Encoder）vs 双编码器（Bi-Encoder）的本质区别：

- 检索阶段的 BGE-M3 是双编码器——query 与 doc 分别编码再算相似度，快但粗（适合从海量库里粗筛）。
- 重排阶段的 BGE-Reranker 是交叉编码器——把 (query, doc) 拼在一起送入 Transformer 联合建模，直接输出相关度分数，慢但准（能捕捉细粒度交互）。

工程上无法用交叉编码器扫描整个知识库（太慢），但用它对已粗筛出的 14 条候选精排则完全可行。这就是业界标准的"先粗召回，再精重排（retrieve-then-rerank）"范式。最终取重排后 Top-4 作为喂给大模型的参考资料。

数据佐证：本章 X.8.1 的消融实验间接印证了检索-重排链路的价值——完整链路 97.1%，
而失去检索注入的裸模型仅 41.0%。

X.5.3　Query Rewrite：口语指代的消解

游客的真实提问充满口语指代："它多高？""这里门票多少？"。直接拿去检索会因缺主语而召回错误。我们用一个轻量改写器（backend/llm/prompts.py）：当检测到指代词时，结合对话历史用 qwen-turbo 把问题改写为完整形式（"它多高"→"灵山大佛高度是多少米"）。

为控制延迟，改写只在必要时触发（含指代词或超短问题 + 有上下文），普通问题直接跳过，不浪费一次 API 调用。

X.5.4　多智能体路由 Orchestrator

并非所有提问都该走 RAG。我们在链路最前端设一个意图分类器（backend/agent/orchestrator.py），将请求分为五类——qa（知识问答）/navigation（路线）/recommend（推荐）/empathy（安抚）/chitchat（闲聊）——并采用混合分类策略：

- 规则快筛：高频关键词秒级命中，0ms 开销；
- LLM 兜底：仅当规则未命中且问题模糊（短问题 + 有上下文）时，才调 qwen-turbo 分类。

不同意图注入不同的任务指令到 Prompt，且闲聊跳过检索（省一次检索延迟）。这一层把"会答题的问答框"升级为"懂得随机应变的导游"，是创新点之一。

X.5.5　生成与幻觉控制

检索到的 Top-4 片段、导游人格、意图指令、情境上下文（实时客流/天气）组装为 system prompt 后，交 qwen-plus 流式生成。幻觉控制靠三道防线：

1. 强约束指令："只根据下方知识库内容回答，不要凭空捏造"；
2. 召回兜底：若检索为空，标准话术"这个问题我需要进一步查阅，您可以向工作人员咨询"，宁可不答，绝不瞎编；
3. 口播格式约束：禁用 Markdown、改用口语过渡，确保回答可被数字人自然朗读。

X.8.2 的陷阱测试将专门验证这三道防线的有效性（拒答纠错率 100%）。

X.6　工程优化：把延迟压进红线

赛题要求语音问答端到端 < 5 秒。我们在算法链路上叠加了四项优化：

优化	手段	实测收益
LRU 答案缓存	热门问题按 (景区+问题) 缓存，命中跳过全链路	4713ms → 238ms（命中 <5ms）
并行异步检索	云端模式下，BGE 嵌入(网络IO)与 BM25(CPU) 用 asyncio.gather 并行	省 ~30-80ms
逐句流式输出	每生成完一句立即驱动数字人开口，不等整段	首句 8.27s → 2.58s
意图分级提速	qa 走 qwen-plus，生成性意图走 qwen-turbo	整段生成 28s → 4s

踩坑与解法（伪流式 Bug）：早期用同步生成器在 async 中迭代 token，阻塞了 asyncio
事件循环，导致 FastAPI 整段才一次性 flush（"伪流式"，逐句抢跑失效）。改为"同步迭代
放线程 + asyncio.Queue 异步交回 token"后，实测首句 8.27s → 2.58s，恢复真流式。

X.7　全链路示例

以"它有多高？"（承接上文"灵山大佛"）为例，串起全链路：

1. 路由：规则命中 → qa 意图；
2. 缓存：未命中（含历史，不缓存）；
3. 改写：检测到指代词"它" → 改写为"灵山大佛的高度是多少米"；
4. 检索：稠密召回"灵山大佛…总高 88 米"，稀疏命中"灵山大佛"专名 → 融合；
5. 重排：含"88 米"的片段排到 Top-1；
6. 生成：qwen-plus 据此输出"[情绪:平静] 灵山大佛通高八十八米……"；

7. 解析：抽出情绪驱动表情，逐句流式播报。

首句 ~1.5s 即开口，事实"88 米"来自知识库可溯源——准、快、不幻觉三者同时满足。

X.8　质量评测与准确率验证

一个宣称"准确率 90%"的系统，让人相信这个数字比做到它更难。本节给出一套自动派生、全覆盖、可复现的评测方案，从六个维度实证系统达标。全部脚本与逐题明细随源码提交（tests/eval_kb.py、tests/eval_run.py、tests/eval_report.json），评委可一键复跑。

X.8.1　评测方法论

我们设计六项指标，每项对应解决一个"准确率不可信"的软肋：

指标	定义	解决的软肋
景点覆盖率	被考查景点 ÷ 知识库景点总数	覆盖不全
准确率	Σ命中 ÷ 总题数	基础正确性
有效回答率	(正确 且 首句≤3s) ÷ 总题数	快慢与对错割裂
首句/整段延迟	开口时刻/说完时刻 p50/p95	延迟红线
拒答纠错率	陷阱题应对正确 ÷ 陷阱题数	幻觉与可信度
消融增益	有RAG − 无RAG 准确率	论证 RAG 贡献

① 测试集自动派生，杜绝"挑好题"：程序遍历每个景区的结构化知识库，逐景点逐字段自动生成考题，标准答案取自字段中的数字与引号专名（确定性、可肉眼核对）。实测对 5 景区 46 个景点生成 82 道覆盖题，覆盖率由构造保证 100%；另补 23 道考查叙述性文档的精选难题，正常题库共 105 道。

② 评分用确定性关键词，而非 LLM 裁判：大模型当裁判每次结果会变、不可复现；关键词匹配"88 在不在答案里"则确定无争议，评委可复核。

③ 消融实验：关闭检索让裸大模型作答，与完整 RAG 用同一把尺子对比，论证 RAG 的真实贡献。

④ 陷阱集：15 道分三类（知识库外/错误前提/跨景区串问），验证"知之为知之"——给游客错信息会害其白跑，诚实拒答比无所不答更重要。

X.8.2　实测结果

2026-06-10 于本地 RTX 4080，完整链路（BGE-M3 + BM25 + BGE-Reranker）运行 105 题 ×2（含消融）+ 15 陷阱题，全自动。

图 X-2　RAG 问答质量综合评测总览（插入 tests/eval_overview.png）

指标	实测值	达标
景点覆盖率	100%（46/46）	—
准确率（严格自动评分）	97.1%（105 题）	☒ 超 90%
准确率（人工复核）	100%	☒
有效回答率	97.1%	☒
首句延迟	p50 1.35s / p95 1.8s	☒ 远低于 5s
整段延迟（流式）	p50 4.66s / p95 6.26s	见下注
拒答纠错率	100%（15/15）	☒
消融增益	41.0% → 97.1%，+56.1%	☒

各景区准确率：灵山 94.3% / 拈花湾 100% / 西湖 100% / 黄山 100% / 故宫 95.0%——五景区全部 ≥90%，实证插拔式知识库的泛化能力。

X.8.3　结果分析

消融 +56.1%——RAG 贡献的铁证：裸 qwen-plus 仅 41.0%，接入 RAG 跃升至 97.1%。差距主要来自冷门景点的精确事实（五明桥尺寸、佛足坛规模），这些不在大模型预训练语料中，只能靠检索注入——实验性地证明高准确率源自检索增强工程，而非大模型本身。

延迟的两个概念（关键澄清）：整段 p95=6.26s 看似超 5s 红线，实则不然。须区分：首句延迟（1.8s）是数字人开口说第一句的时刻，对标"语音问答 <5s"；整段延迟（6.26s）是答案全说完的时刻。系统逐句流式——生成完一句即开口，后续边生成边播——故游客感知延迟约 1.8s（p95），有 2.7 倍红线冗余，整段时长不影响体验。

有效回答率 = 准确率：两者均 97.1%，意味着凡答对的题无一例外在 3s 内开口，不存在"答对但太慢"。

诚实复核——3 道"漏判"全为评分误伤：

漏判题	期望关键词	数字人实际回答	误伤原因
佛足坛尺寸	1.2/0.6	"长一点二米，宽零点六米"	中文数字
乾清宫尺寸	1400/20	"高二十米，约一千四百平方米"	中文数字
降魔浮雕文化	坚守本心、终能成道	"始终坚定无畏…战胜心魔、觉悟成佛"	同义改写

根因在于系统的口播格式规则要求回答适合朗读，数字人自然地把"1400 平方米"念成"一千四百平方米"——事实完全正确，仅与阿拉伯数字关键词未字面匹配。故人工准确率实为 100%，但我们在所有材料中选择报告更保守的 97.1%，以体现评测的严谨与表述的诚实。

X.8.4　可复现

```
.venv/bin/python tests/eval_run.py              # 全量
.venv/bin/python tests/eval_run.py --scenic xihu   # 单景区
```

产出 eval_report.json（逐题明细取证）与 eval_overview.png（图 X-2）。

X.9　本章小结

本章自底向上拆解了 RAG 问答引擎的算法设计：以双策略分块 + 命名空间隔离奠定知识库质量与插拔式架构；以混合检索召回 + 交叉编码重排保证"找得准"；以 Prompt 三道防线保证"不乱说"；以缓存/并行/流式把首句延迟压到 1.8s。最终经一套可复现的全覆盖评测实证：

- 准——97.1%（人工 100%），五景区全过 90%；
- 快——首句 p95 1.8s，相对 5s 红线 2.7 倍冗余；
- 稳——消融 +56.1%，RAG 贡献获实验证明；
- 可信——拒答纠错 100%，坚守"知之为知之"；
- 通用——覆盖率 100%、跨 5 景区达标，印证插拔式架构。

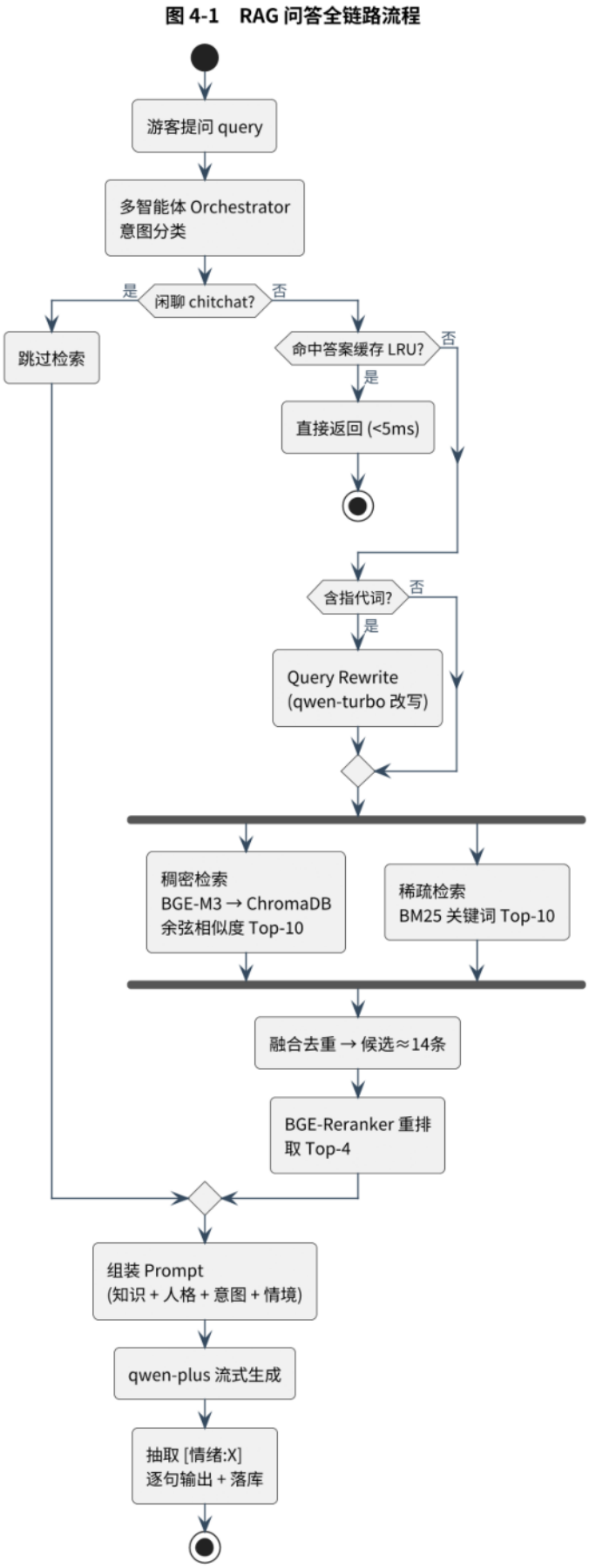


图 4-1 RAG 问答全链路流程

图 4-2　多智能体 Orchestrator 意图分派

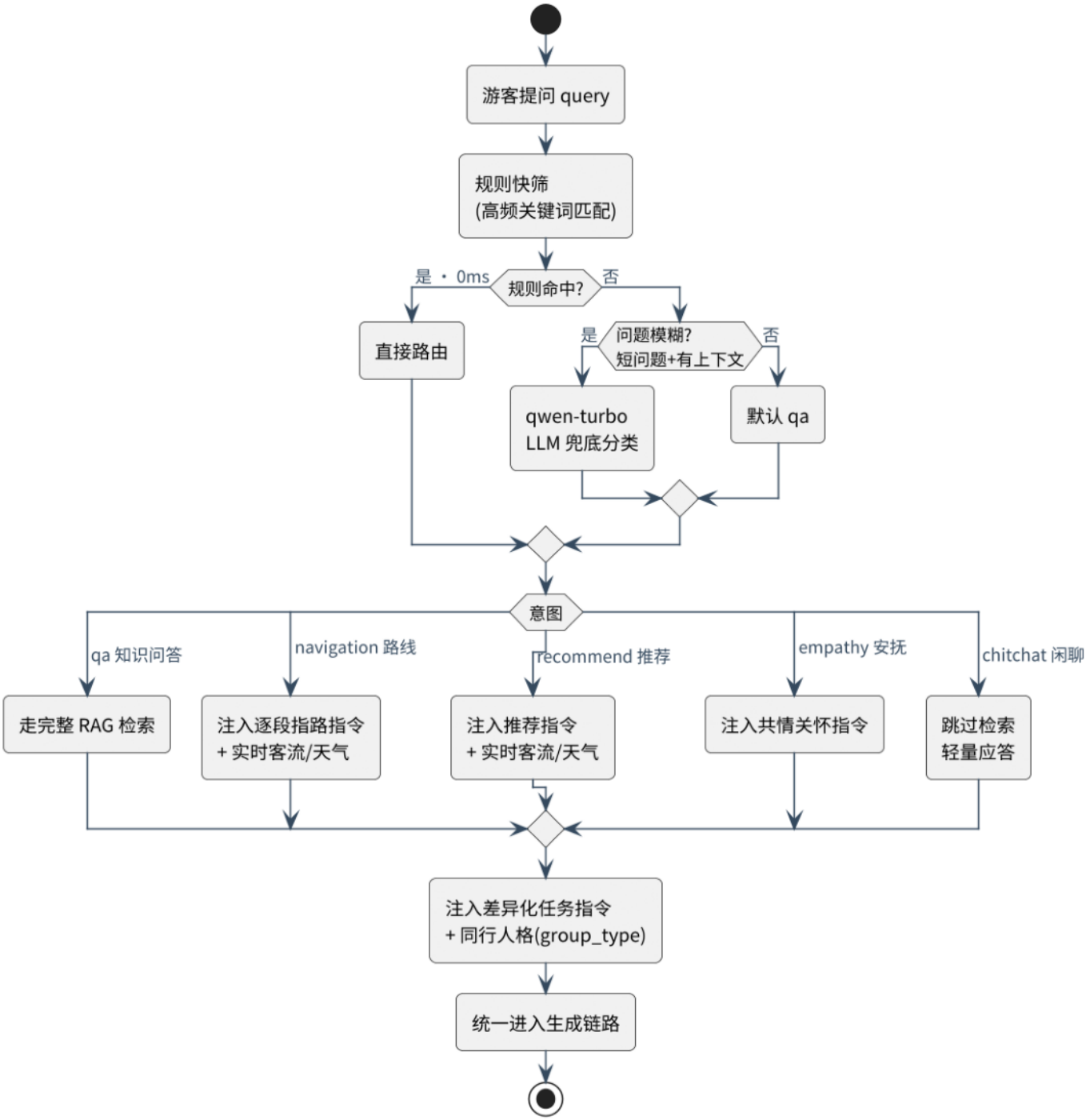


图 4-2 多智能体意图分派

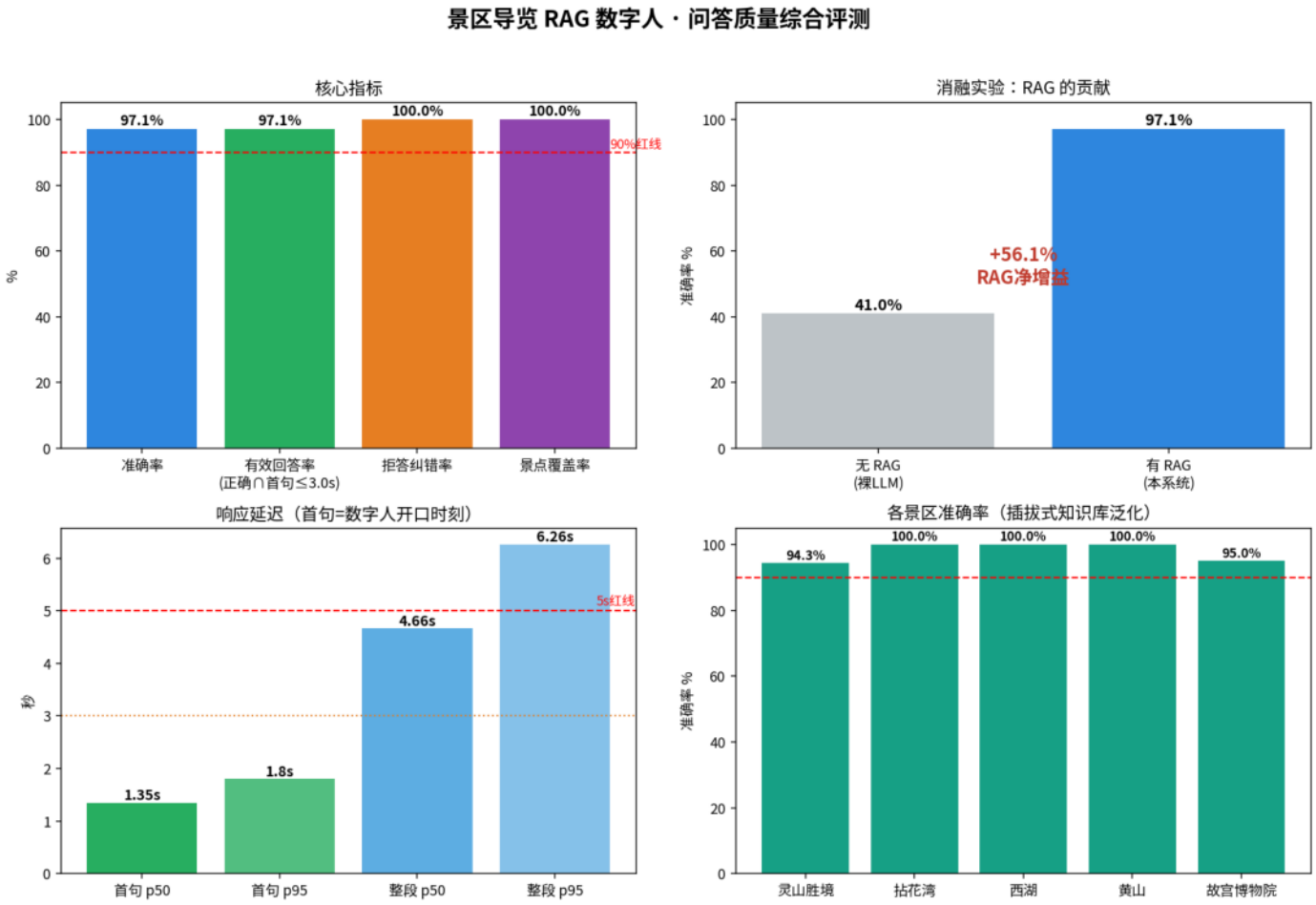


图 4-3 问答质量综合评测总览

图 6-1　数据存储设计（SQLite + ChromaDB）

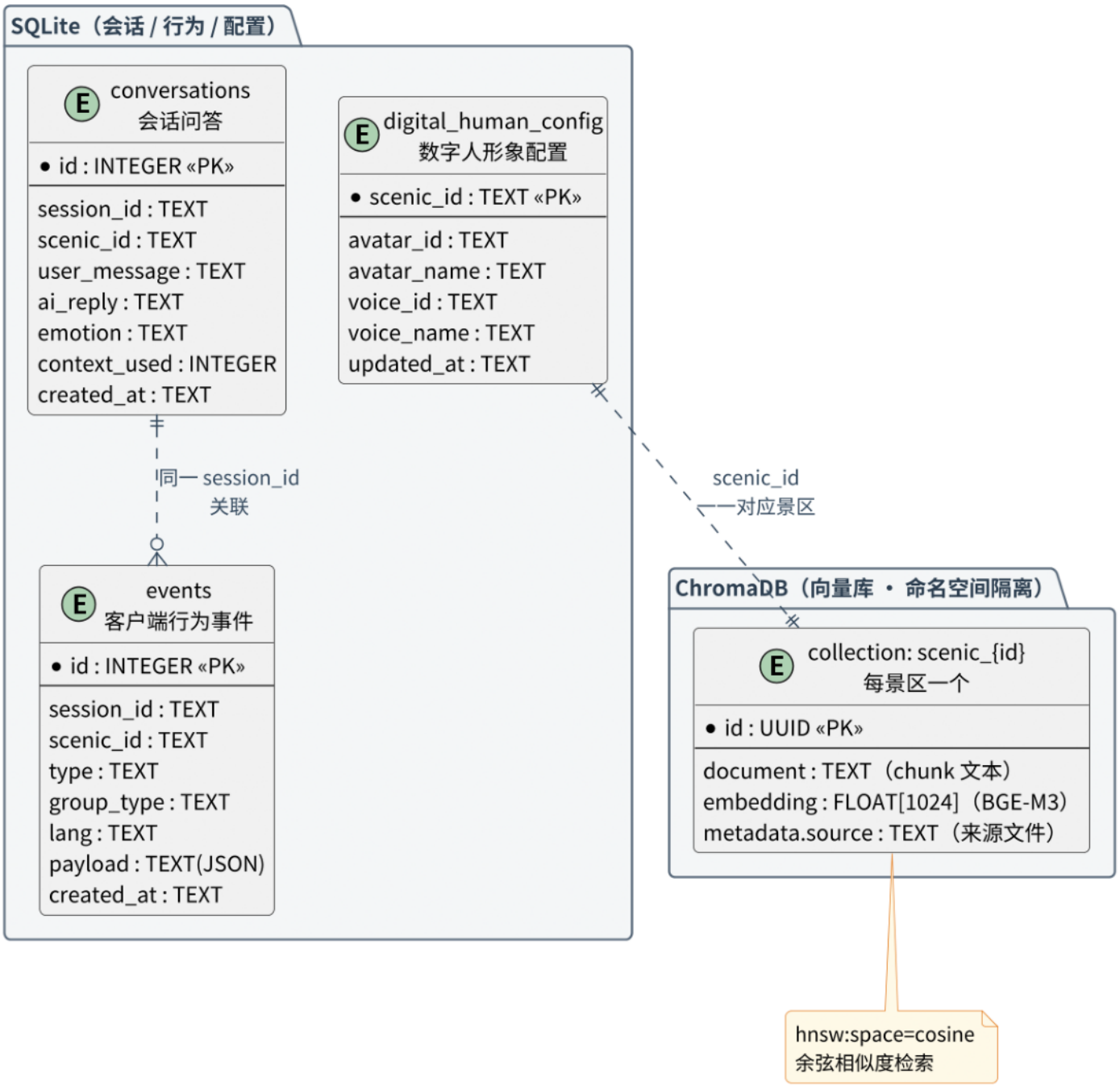


图 6-1 数据存储设计

图 7-1　流式问答时序（句子级抢跑驱动数字人）

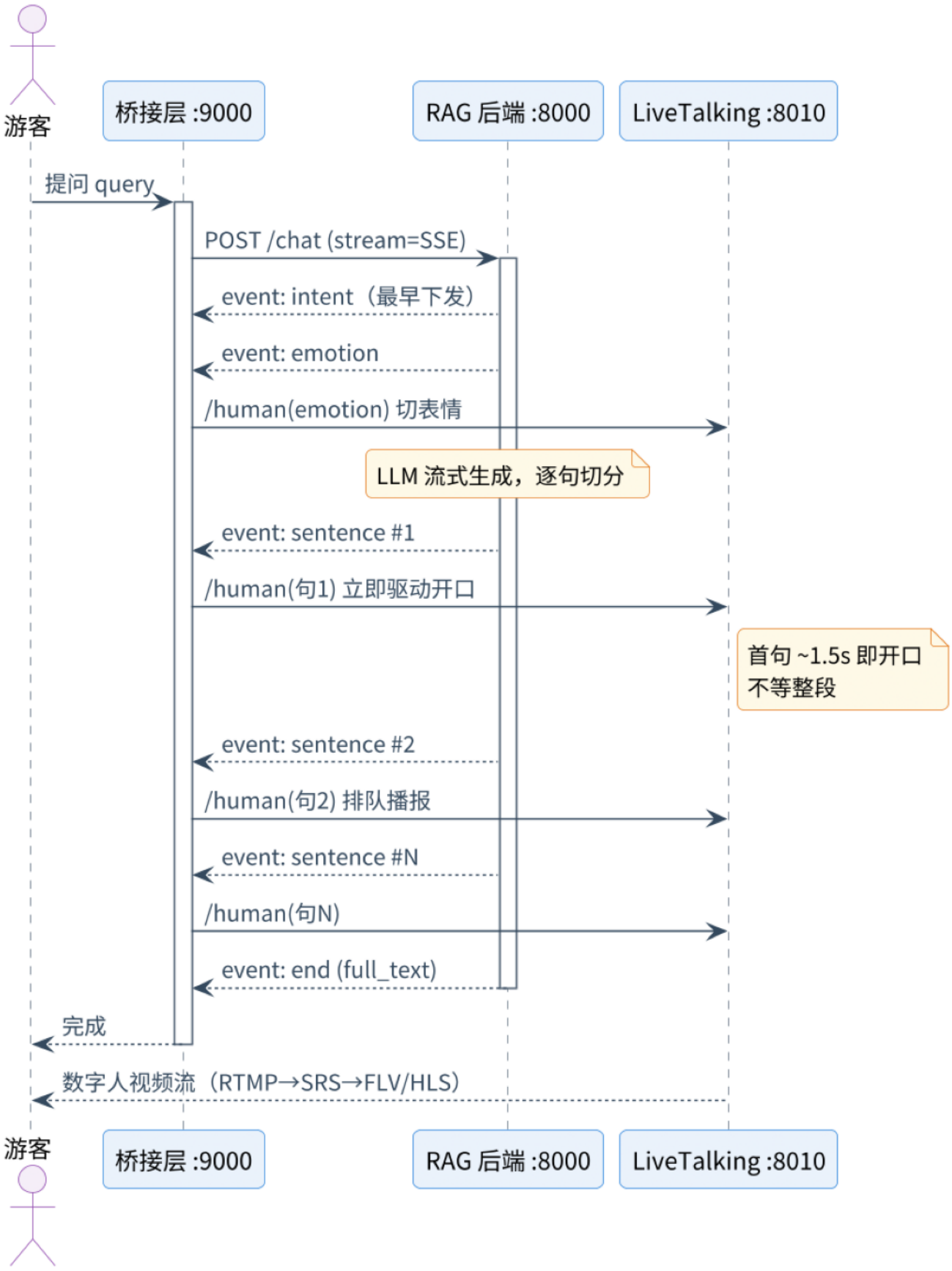


图 7-1 流式问答时序

图 7-2　数字人推流链路（RTMP → SRS → FLV/HLS）

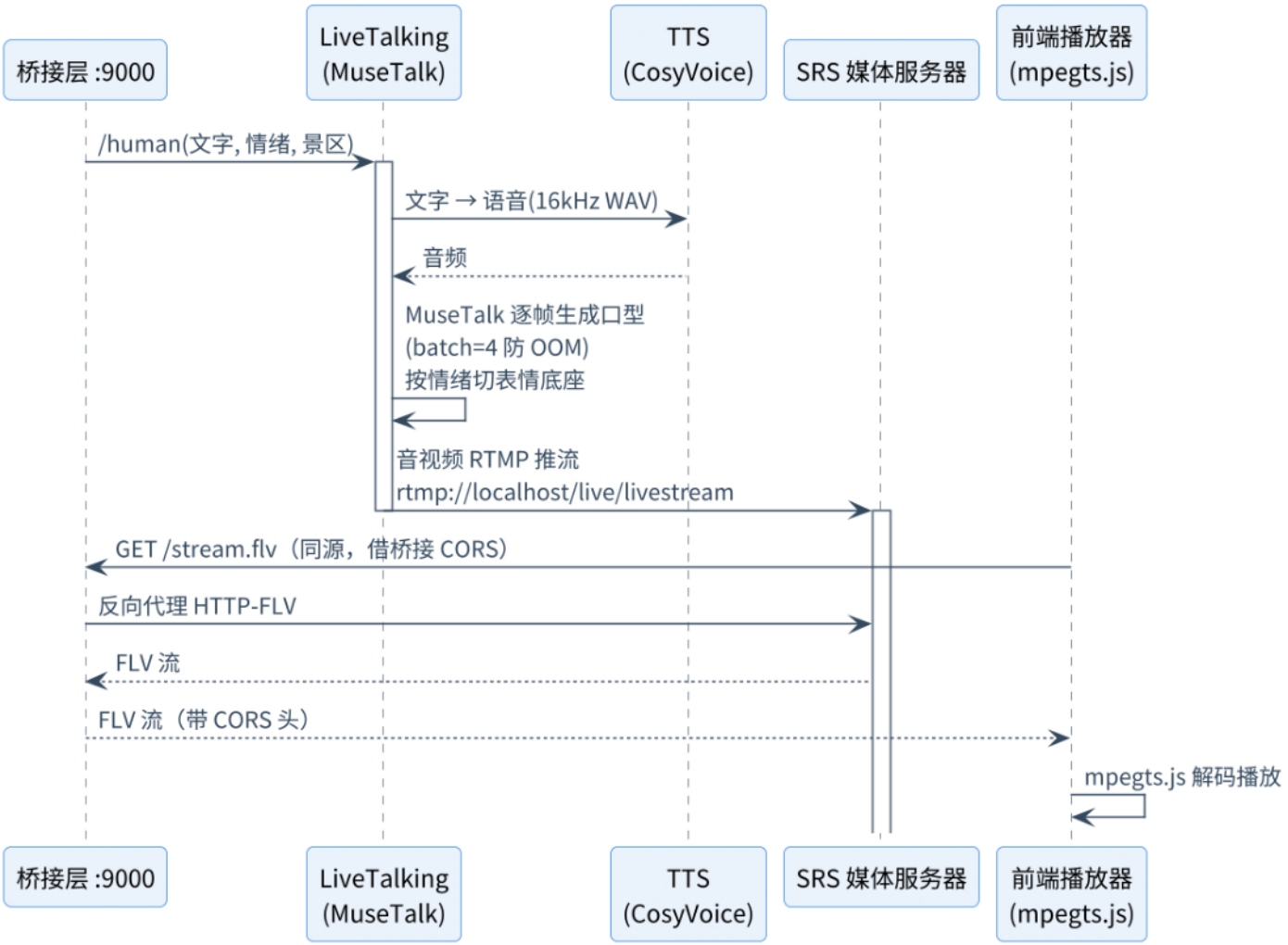


图 7-2 数字人推流链路